DIY Challenge Repo Usage Guide

Living document for how to build, source, and run the robotics challenge software in this repository. Update this guide as the software stack grows.

1. What This Repo Is For

This repository is the active development workspace for DIY Robot Challenge 2026. It combines the new challenge-specific bringup with reused legacy modules and third-party ROS 2 packages.

- `src/challenge_bringup` - current challenge startup package
- `src/diy_motor_control_legacy` - reused legacy motor package from Betsybot
- `third_party_ws` - ROS 2 source packages and external dependencies being integrated

2. Current Reuse Baseline

> **Already reusable now:** legacy drive/control logic, joystick launch pattern, Nav2 bringup skeleton, and collision monitor configuration.

See the detailed reuse review in `docs/reuse_plan_step1.md`.

3. Environment Setup

Open a new terminal and source the challenge environment in this order:

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/src/DIY-Challenge-Repo/third_party_ws/install/setup
source ~/ros2_ws/src/DIY-Challenge-Repo/install/setup.bash
```

If you rebuild one workspace, source it again before launching nodes.

4. Build the Repo

4.1 Build first-party challenge packages

From the repo root:

```
cd ~/ros2_ws/src/DIY-Challenge-Repo
source /opt/ros/humble/setup.bash
colcon build --symlink-install
```

4.2 Build third-party packages

From the repository-local third-party workspace:

```
cd ~/ros2_ws/src/DIY-Challenge-Repo/third_party_ws
source /opt/ros/humble/setup.bash
colcon build --executor sequential --parallel-workers 1 --symlink-ir
```

Use sequential, low-parallel builds when the terminal or swap is under pressure.

5. Launch the Current Challenge Baseline

The bootstrap launch is:

```
ros2 launch challenge_bringup challenge_master.launch.py
```

Useful toggles while the stack evolves:

```
ros2 launch challenge_bringup challenge_master.launch.py use_nav2:=t
```

6. Current Topic Wiring

- Joystick teleop publishes to `/cmd_vel_joy`
- Nav2 will eventually publish to `/cmd_vel_nav`
- Safety output should become `/cmd_vel_safe`
- Hesai LiDAR input for navigation is `/hesai/points`

7. Recommended Development Flow

1. Reuse existing code only where it fits the SAD and safety model.
2. Build new nodes around a clear command chain: joystick/autonomous input, safety gate, motor output.
3. Keep launch files small and focused so they can be composed later.
4. Update this guide whenever a package is added, renamed, or replaced.

8. Next Components to Add

- `diy_cmd_vel_mux` for mode arbitration
- `diy_estop_controller` for hardware-safe stop handling
- RTK + EKF launch set from the SAD
- FAST-LIO2 runtime localization integration
- Mission state machine and obstacle behavior packages

9. Update Policy

This guide is intentionally living documentation. When a new package is added or the launch flow changes, update this file and re-export the PDF so the docs folder stays current.